Buổi 4

State Management và lựa chọn NgRx

Local UI · feature · URL · session · server state · SignalStore · Global Store

Mục tiêu buổi học

Sau buổi này, học viên có thể:

1. Phân loại state: **local UI, feature, URL, session, server** theo phạm vi, vòng đời, số nơi ghi
2. Xây **SignalStore** (`withState`, `withComputed`, `withMethods`, `rxMethod`, `withHooks`) và đặt đúng **phạm vi DI**
3. Dùng **URL làm nguồn sự thật** cho bộ lọc: F5, chia sẻ link, Back/Forward
4. **Làm mới server state sau thao tác ghi**, không gọi API trùng
5. **Xóa state khi đổi phiên**; so sánh SignalStore với Global Store (action, reducer, selector)

Sản phẩm đầu ra: ma trận lựa chọn state · ADR giải pháp · test tính nhất quán của bộ lọc

Lịch buổi học (4 giờ)

Thời gian	Hoạt động
20'	Review buổi 3 (bảng chọn operator), phân tích use case
40'	Loại state, ma trận, SignalStore, URL state, server state, session, Global Store
30'	Live demo: F5 mất bộ lọc, Back sai, rò dữ liệu khi đổi phiên
10'	Nghỉ
120'	Lab 5 bài
20'	Review chéo, chốt ADR-0003

Use case: bốn lời phàn nàn từ CSKH

#	Lời phàn nàn	Đang ở đâu
1	"Lọc phiếu *Đã xử lý*, bấm F5 là mất bộ lọc"	`signal` trong `TicketListPage`
2	"Gửi link cho trưởng nhóm, họ mở ra danh sách khác"	Bộ lọc không có trên URL
3	"Bấm Back thì văng khỏi trang thay vì về bộ lọc trước"	Không có mục lịch sử cho bộ lọc
4	"Tạo phiếu xong quay về, bộ lọc biến mất"	Trang tạo phiếu `navigate(['/tickets'])`

Và một lỗi nghiêm trọng hơn (điểm bắt đầu `session04-start` có sẵn tính năng **Thuê bao xem gần đây**):

> agent1 xem thuê bao, đăng xuất. supervisor1 đăng nhập trên cùng tab → thấy ngay danh sách khách hàng agent1 vừa tra cứu. Dữ liệu cá nhân rò sang người khác.

Năm loại state

Loại	Ví dụ	Mất khi	Công cụ thường dùng
Local UI	Dropdown mở, nút đang quay, thông báo	Đóng component	`signal` trong component
Feature	Kết quả + trạng thái tải của một màn hình	Rời màn hình	SignalStore ở component/route
URL	Bộ lọc, trang, sort, id đang xem	Theo URL	Query/path params
Session	Người đăng nhập, dữ liệu gắn người dùng	**Đổi phiên**	Store `providedIn: 'root'` + xóa khi đổi phiên
Server	Danh sách phiếu, chi tiết thuê bao	Khi server đổi	`rxResource` / store; làm mới sau ghi

> Server state không phải "của mình": nó là **bản sao** có thể cũ. Câu hỏi luôn là: khi nào làm mới?

Ma trận lựa chọn: hỏi trước, chọn công cụ sau

Câu hỏi	Nếu trả lời là…	Thì nghiêng về
Ai cần đọc?	Một component	`signal`
	Cả màn hình, nhiều component	SignalStore ở component/route
	Người khác (qua link)	**URL**
Mất khi nào?	Rời màn hình	Store ở component
	Đăng xuất	Store root + **xóa khi đổi phiên**
Mấy nơi ghi?	Nhiều nơi không liên quan	Cân nhắc Global Store
Giữ khi F5 / chia sẻ?	Có	URL (hoặc storage nếu chỉ một người)
Cần truy vết?	Audit, time-travel	Global Store + DevTools

Bản áp dụng cho Care Portal: `docs/ma-tran-state.md` (Bài 1).

SignalStore: bốn khối

```
export const TicketListStore = signalStore(
  withState<TicketListState>({ items: [], total: 0, status: 'loading' }),   // state gốc
  withComputed(({ filters }, auth = inject(AuthService)) => ({                 // suy ra
    query: computed(() => ({ status: filters().status, priority: filters().priority,
                             assignee: filters().onlyMine ? auth.username() : null })),
  })),
  withMethods((store, tickets = inject(TicketPort)) => ({                      // hành động
    setFilters(change: Partial<TicketFilters>) { ... },
    reload() { ... },
  })),
  withHooks({ onInit(store) { store._load(computed(() => ({ ... }))); } }),   // vòng đời
);
```

- `patchState(store, ...)` là cách **duy nhất** ghi state: không ai gán trực tiếp
- Thành viên bắt đầu bằng `_` là **riêng tư**: component không gọi được `store._load`

Tải dữ liệu trong store: `rxMethod`

```
const load = rxMethod<{ query: TicketQuery; page: PageRequest }>(
  pipe(
    tap(() => patchState(store, { status: 'loading' })),
    switchMap(({ query, page }) =>                          // buổi 3: hủy request cũ
      tickets.search(query, page).pipe(
        tap(({ items, total }) => patchState(store, { items, total, status: 'done' })),
        catchError(() => { patchState(store, { status: 'error' }); return EMPTY; }),  // trong
      ),
    ),
  ),
);

// onInit: truyền SIGNAL → mỗi lần query/trang đổi là tải lại
store._load(computed(() => ({ query: store.query(), page: store.page() })));
// reload(): truyền GIÁ TRỊ → tải một lần nữa với đúng query hiện tại
load({ query: store.query(), page: store.page() });
```

Phạm vi DI quyết định vòng đời store

Khai báo	Sống	Dùng cho
`signalStore({ providedIn: 'root' }, ...)`	Cả app	Session state: phải **tự xóa khi đổi phiên**
`providers: [Store]` trên **route**	Route và route con	State dùng chung giữa nhiều màn hình con
`providers: [Store]` trên **component**	Component	State của một màn hình (Care Portal)

```
@Component({ selector: 'care-ticket-list-page', providers: [TicketListStore], ... })
export class TicketListPage {
  protected readonly store = inject(TicketListStore);
}
```

> Store theo component: rời màn hình là mất, quay lại là tải mới. Không có state cũ, không phải tự dọn.

Test store không cần component

```
TestBed.configureTestingModule({ providers: [
  provideRouter([]), TicketListStore,
  { provide: TicketPort, useValue: port },                    // fake từ buổi 1
  { provide: AuthService, useValue: { username: signal('agent1') } },
]});
const store = TestBed.inject(TicketListStore);
TestBed.tick();                                               // chạy effect của rxMethod

await store.setFilters({ status: 'RESOLVED', onlyMine: true });
TestBed.tick();
expect(port.calls.at(-1)).toEqual({
  query: { status: 'RESOLVED', priority: null, assignee: 'agent1' },
  page: { page: 0, size: 50, sort: 'id,desc' },               // về trang đầu, giữ kích thước
});
```

`TestBed.tick()` thay cho `flushEffects()` cũ: chạy các effect đang chờ.

URL là nguồn sự thật

❌ Hai nguồn, đồng bộ hai chiều

```
effect(() => router.navigate([], {
  queryParams: { status: status() } }));
route.queryParamMap.subscribe(p =>
  status.set(p.get('status')));
```

Dễ vòng lặp, lệch nhau, khó test

✅ Một nguồn: URL

```
filters = computed(() =>
  filtersFromParams(params()));

setFilters(change) {
  return router.navigate([], {
    relativeTo: route,
    queryParams: toQueryParams(...),
  });
}
```

Mọi thay đổi (chọn bộ lọc, Back/Forward, mở link, F5) đều đi qua **một đường**: URL → `filters` → tải lại.

Đọc URL: không tin input

```
export function filtersFromParams(params: ParamMap): TicketFilters {
  return {
    status: oneOf(params.get('status'), STATUSES),       // ngoài danh sách → null
    priority: oneOf(params.get('priority'), PRIORITIES),
    onlyMine: params.get('mine') === '1',
  };
}

export function pageFromParams(params: ParamMap): PageRequest {
  // page trên URL đếm từ 1; size chỉ 20/50/100; sort chỉ cột cho phép
  ...
}
```

URL	Hiểu thành
`?status=RESOLVED&page=2`	Đã xử lý, trang thứ 2
`?status=DROP&size=5000&sort=password,asc`	Không lọc, 20 dòng, sort mặc định

Giá trị mặc định không ghi lên URL: `/tickets?status=NEW` thay vì `…&page=1&size=20&sort=id,desc`.

Back/Forward và tải lại cùng query

- Mỗi `router.navigate` (không `replaceUrl`) là **một mục lịch sử** → Back quay về bộ lọc trước
- Back/Forward là sự kiện `popstate` → router đổi URL → `filters` đổi → `rxMethod` tải lại
- **Bấm "Thử lại" khi URL không đổi:** router bỏ qua điều hướng tới cùng URL → phải `store.reload()`

```
await store().setFilters({ priority: 'HIGH' });
expect(router.url).toBe('/tickets?status=RESOLVED&priority=HIGH');

await afterNavigation(() => location.back());
expect(router.url).toBe('/tickets?status=RESOLVED');
expect(lastCall()?.query.priority).toBe(null);
```

> Test với `RouterTestingHarness` + `provideLocationMocks()`. Nhớ gọi `router.initialNavigation()`: không có nó router không nghe popstate.

Làm mới server state sau thao tác ghi

Sau khi…	Làm gì	Vì sao
Duyệt phiếu (trên cùng màn hình)	`store.reload()` : **đúng query + trang**, một lần	Server đã đổi; giữ chỗ người dùng đang xem
Tạo phiếu (màn hình khác)	Quay về **đúng URL danh sách** qua `returnUrl`	Danh sách là store mới → tải mới → thấy phiếu vừa tạo
Gặp 409	Không tự tải lại; người dùng bấm "Tải lại"	Buổi 2

```
// list → create: mang theo URL danh sách hiện tại
<a routerLink="/tickets/new" [queryParams]="{ returnUrl: store.listUrl() }">Tạo phiếu</a>

// create → list: chỉ chấp nhận /tickets..., chống open redirect
function backToList(url: unknown): string {
  const safe = safeReturnUrl(url);
  return safe === '/tickets' || safe.startsWith('/tickets?') ? safe : '/tickets';
}
```

Session state: phải xóa khi đổi phiên

```
export const RecentSubscribers = signalStore(
  { providedIn: 'root' },                              // sống cả app
  withState<{ items: RecentSubscriber[] }>({ items: [] }),
  withMethods((store) => ({ add(s: RecentSubscriber) { ... } })),
  withHooks((store) => {
    const auth = inject(AuthService);
    return {
      onInit() {
        effect(() => {                                 // effect: đồng bộ theo sự kiện bên ngoài (đổi phiên)
          auth.username();
          untracked(() => patchState(store, { items: [] }));
        });
      },
    };
  }),
);
```

Mọi state `providedIn: 'root'` chứa dữ liệu gắn với người dùng đều cần câu trả lời: "Đăng xuất thì ai xóa nó?". Không có câu trả lời = rò dữ liệu.

Global Store (NgRx Store): cùng tính năng thì sao?

```
// actions
export const ticketListActions = createActionGroup({ source: 'Ticket list', events: {
  'Filters changed': props<{ filters: TicketFilters }>(), 'Page changed': props<{ page: PageRequest }>(),
  'Load succeeded': props<{ items: TicketSummary[]; total: number }>(), 'Load failed': emptyProps(),
}});
// reducer
export const ticketListFeature = createFeature({ name: 'ticketList', reducer: createReducer(initial,
  on(ticketListActions.filtersChanged, (s, { filters }) => ({ ...s, filters, page: { ...s.page, page: 0 } })),
  on(ticketListActions.loadSucceeded, (s, { items, total }) => ({ ...s, items, total, status: 'done' })), ...)});
// effect
load$ = createEffect(() => this.actions$.pipe(ofType(filtersChanged, pageChanged),
  concatLatestFrom(() => this.store.select(selectQuery)), switchMap(([, q]) => this.tickets.search(...).pipe(
    map((r) => loadSucceeded(r)), catchError(() => of(loadFailed()))))));
// selector
export const selectQuery = createSelector(selectFilters, selectUsername, (f, u) => ({ ... }));
```

SignalStore hay Global Store?

	SignalStore	Global Store (Store + Effects)
Lượng code cho trang phiếu	1 file ~140 dòng	4–5 file (actions, reducer, selectors, effects, đăng ký)
Phạm vi	Component / route / root	Toàn app (feature state đăng ký vào store gốc)
Truy vết	Không có sẵn	Mỗi thay đổi là một action, Redux DevTools time-travel
Ai được ghi	Method của store	Chỉ reducer, qua action
Hợp với	State theo màn hình, team nhỏ–vừa	Nhiều màn hình cùng ghi một state, cần audit, nhiều team cần quy ước chặt

> Care Portal hiện tại: SignalStore + URL + session store là đủ. Ghi lý do và điều kiện xem lại vào ADR-0003.

Live demo (30 phút)

1. Trang Phiếu, lọc **Đã xử lý**, bấm **F5** → mất bộ lọc. Copy URL gửi "trưởng nhóm" (cửa sổ khác) → danh sách khác
2. Lọc 3 lần liên tiếp, bấm **Back** → rời khỏi trang
3. agent1 mở 2 thuê bao, quay về trang Thuê bao: dòng **Xem gần đây**. Đăng xuất, đăng nhập `supervisor1` → **vẫn thấy** 2 thuê bao của agent1
4. Chuyển sang nhánh `session04-end`, làm lại 1–3: URL `/tickets?status=RESOLVED`, F5 giữ, Back về bộ lọc trước, supervisor1 không thấy gì

> Hỏi lớp sau bước 3: "Trong dự án của anh chị, còn service `providedIn: 'root'` nào giữ dữ liệu người dùng?"

Lab (120 phút)

Bài	Nội dung	Thời gian
1	Ma trận lựa chọn state cho Care Portal (`docs/ma-tran-state.md`)	15'
2	`TicketListStore` : state, computed, methods, `rxMethod` ; test store	35'
3	URL là nguồn sự thật: đọc/ghi URL, Back/Forward, reload; test nhất quán; ADR-0003	35'
4	Làm mới sau ghi: duyệt → reload đúng chỗ; tạo phiếu → về đúng bộ lọc	15'
5	Xóa state khi đổi phiên: `RecentSubscribers` thành SignalStore root có hook	20'

Điểm bắt đầu: `git checkout session04-start` · Đề bài: `course/session-04/lab/README.md`

Bẫy hay gặp khi làm lab

Hiện tượng	Nguyên nhân
Đổi bộ lọc, API gọi 2 lần	`setFilters` và `setPage` cùng điều hướng; nz-table phát lại cùng trang → so `samePage`
`NG0203` trong store	`inject` / `toSignal` gọi trong method thay vì trong `withProps` / `withComputed`
Test store không thấy request	Quên `TestBed.tick()` sau khi khởi tạo/đổi state
Back trong test không làm gì	Thiếu `router.initialNavigation()` hoặc không chờ `NavigationEnd`
URL `?page=0` hiện "trang 0"	Trộn trang đếm từ 0 (code) và từ 1 (URL)
Tạo phiếu xong về trang chủ	`safeReturnUrl` trả `/` khi URL không hợp lệ → cần mặc định `/tickets`
supervisor1 vẫn thấy thuê bao cũ	Kiểm tra bằng `page.goto` (tải lại cả trang) thay vì điều hướng trong app

Tổng kết

1. Phân loại state trước, chọn công cụ sau: **phạm vi, vòng đời, số nơi ghi, chia sẻ, truy vết**
2. **SignalStore**: `withState`, `withComputed`, `withMethods` (`rxMethod`), `withHooks`; `patchState` là cách duy nhất ghi
3. **Phạm vi DI = vòng đời**: store theo component tự dọn; store root phải tự xóa khi đổi phiên
4. **URL là nguồn sự thật** cho bộ lọc; đọc URL như đọc input người dùng
5. Server state là bản sao: **làm mới sau ghi**, đúng chỗ, một lần
6. Global Store khi cần truy vết và quy ước chặt, không phải mặc định

Buổi 5: Hiệu năng và quản lý bộ nhớ: đo trước khi sửa, virtual scroll, heap snapshot.